Data Transparency

Individual text documents are no longer the sole/primary means of disseminating academic scholarship: developments influencing the emergence of "Executable Research Objects"

Replication Crisis — Difficulty in reproducing both older, influential studies and new ones

Data Transparency becomes a priority — Wider access to raw data can — hopefully! — help clarify why replication is a problem

MIBBI and friends — Minimum Information for Biological and Biomedical Investigations

"FAIR" — Findable, Accessible, Interoperable, Reusable

Open-Access Data Sets — Raw data files are usually shared according to different copyright and availability protocols than academic/scientific publications themselves

Research Object Specifications — Research Object "Bundle"; Executable Research Objects

Data Transparency

This group of slides includes my original deck for the JATS-Con presentation interleaved with text commentary I composed as notes for while speaking (or "textovers"). For reading, I think it works well to merge them together.

I assume here that I do not need to spend a lot of time introducing concepts like Data Transparency and open-access data sets. Long story short, increasingly the basic unit of dissemination for academic scholarship and scientific research is not the individual text publication, but a package that can contain data, text, computer code, and multimedia presentations.

Surely there are multiple factors that have influenced this development over the past decade or two, but there is no question that concerns about reproducing research have played a role.

"Replication Crisis"

- This phrase began to be used in the 2010s
- Some "failures to reproduce" derive from statistical or computational errors
- Controlled experimentation is particularly questionable vis-à-vis social sciences and psychology
- "Chaos effect": subtle differences in setup and parameters can yield significant differences in results

"Replication Crisis"

In particular, the "Replication Crisis" that became recognized in the 2010s has changed the way scientists seek to evaluate new findings. Instead of publications just providing a summary overview, contemporary scientists want to learn about the research in greater depth: to examine raw data files, data acquisition protocols, data-transformation pipelines, and analytic methods.

Rather than accepting claims at face value from peer-reviewed articles, scientists seek to vet new research more thoroughly than in the past, especially in light of the replication crisis. After all, classic studies affected by the crisis were subject to peer review and usually had no obvious methodological flaws — they were conducted by competent, sometimes highly influential researchers.

Data Transparency Becomes a Priority

- Publishers may require authors to share raw data or else explain specifically why it is not possible to do so
- Funding agencies may require data transparency as a precondition for grantees (e.g., the Bill and Melinda Gates Foundation)
- "Data Availability" or "Supplemental Materials" sections are now common on publication landing pages

Authors however often confuse tables or summaries with actual data sharing — i.e., publishing raw data sets (not just a statistical overview)

Data Transparency Becomes a Priority

In response to a growing realization that even well-executed investigations could yield questionable and non-reproducible results, some publishers have started to *require* that authors provide raw data alongside their articles, or else to explicitly state why they are unable to do so (assuming their research involves empirical data sets to begin with).

Similar policies have also been adopted by funding agencies. For example, the Bill and Melinda Gates foundation requires that all grantees "include a Data Availability Statement (DAS) describing where the underlying data can be found ... Underlying data includes all primary data, metadata, and any additional information needed to understand, assess, and replicate the study findings."

→ Gates Foundation 2025 Open Access Policy: Data Availability Guide
https://openaccess.gatesfoundation.org/wp-content/uploads/2025/03/Grantee-DAS-Guide.pdf

MIBBI and Other Minimum Information Standards

Minimum Information for Biological and Biomedical Investigations

- Standard for describing experiments, lab, equipment, and data acquisition
- Employs structured metadata in lieu of text descriptions
- MIBBI encompasses almost 40 specific formats, such as:
  - Minimum Information About a Microarray Experiment (MIAME)
  - Minimum Information About a Proteomic Experiment (MIAPE)
  - Minimal Information Required In the Annotation of Models (MIRIAM)
  - Minimum Information About a Simulation Experiment (MIASE)
  - Minimum Information about a Flow Cytometry Experiment (MIFlowCyt)
  - Minimum Information about a Functional Genomics Experiment (MIAFGE)
  - Core Information for Metabolomics Reporting (CIMR)
  - Proteomics Identifications Database (PRIDE)

MIBBI and Other Minimum Information Standards

Data Transparency goes beyond just sharing data once it is acquired. For scientific fields where observations are obtained from physical or laboratory equipment, properly assessing and reproducing research requires relatively detailed understanding of the experimental setup prior to data being captured. Minimum Information standards — also sometimes called "Minimum Information Checklists" (MICs) — allow these details to be notated in a structured manner conformant to domain-specific templates. MIBBI, for example, was formally established in 2008, and its various guidelines help authors correctly share details about their work prior to publication. By extension, the overall procedures through which samples and information are transformed — both in the physical and digital/computational realms — should be documented with an eye toward replication.

"Findable, Accessible, Interoperable, Reusable"

- FAIRdata: Standard for how data is organized and presented
- FAIRsharing: Standard for how data is published and deposited
- Accessibility: Beyond just raw data, minimizing reliance on special software or computing environments — prefer open-source
- Reusability: Effort should be made to help future scientists analyze the raw data — share code to give future researchers a head start

"Findable, Accessible, Interoperable, Reusable"

According to the popular "FAIR" standard, data should be Findable, Accessible, Interoperable, and Reusable. Arguably the most important consideration here is accessibility. In the typical case, a researcher will learn about some scientific work or experiment from peer-reviewed literature. Once they are reading a specific publication — and should they wish to examine the research in greater detail — the question then becomes how readily they can acquire the associated raw data and examine it usefully.

Interoperability means that multiple data sets can be studied and analyzed side-by-side, and Reusability ensures that data and code could be utilized again in new experiments — reproducing and/or extending the original data set. But accessibility is a precondition for everything that follows.

Data Sets

- At a minimum, data sets are *citeable* (independently referenced) collections of one or more raw data files
- Data sets are often published outside any paywall, with a more permissive licence than accompanying papers — not an ideal solution
- For FAIRsharing, raw data sets should be accompanied by computer code — it usually requires special programming to deserialize, manipulate, and do useful things with the raw data
- Don't assume users can load raw data with external software (e.g., CSV often doesn't work well just viewed as a spreadsheet)

Data Sets

Accessibility has multiple dimensions. Starting from a published manuscript, there are usually several steps required to download, deserialize, and study data sets. With FAIR data, these steps can hopefully be accomplished without relying on external software or manual processing. Ideally, computer code needed to unpack and visualize data contents are included as part of the data set.

Those publishing FAIR data should not assume that users will have external software available, especially software that is not free and open-source. Researchers should be able to assess scientific work without having access to the same applications or tools that the original scientists employed. For most data sets, raw data needs special deserialization and visualization tools. In terms of FAIR Accessibility, for example, XML files can't just be viewed as DOM trees in web browsers, and CSV files cannot just be opened as spreadsheets.

Executable Research Objects

- A way to achieve FAIR data transparency: data, code, and multimedia bundled together
- Research Object "Bundles" follow a constrained folder layout and required JSON manifest, with other recommended metadata
- Executable Research Objects are less formally defined, but with an emphasis on code sharing

Other related terms are "Research Compendia", "Story Maps" (in ArcGIS), "Data Infrastructure Building Blocks" (DIBBs), "Open Data Cube" (ODC), or "The Whole Tale" (a DIBBS initiative)

Executable Research Objects

As a standard related to FAIR data, the Research Object specification includes formats such as Research Object Bundles and RO-Crate that govern how data packages are organized and what kinds of metadata they provide. These formats typically include one or more structured files that encapsulate a machine-readable summary of data contents, plus raw data files organized for straightforward access.

More generally, "Research Objects" can refer to any data set published according to FAIR principles — so we can use the term more informally — but the common theme is that raw data is supplemented with metadata and code to facilitate accessibility, interoperability, and reuse. An "Executable" Research Object prioritizes effective design for included source code. In particular, data sets may be developed as self-contained computer applications with a dedicated entry-point that supplies access to different parts and facets of the Research Object — so the Research Object's code is unified into a single executable package. This promotes accessibility because it gives users an obvious next step to take after downloading the data set.

Publications Within Research Objects

Or so we might predict:

In the future ...

- Executable Research Objects will encompass text documents as well as code and data
- Readers may first discover publications as PDF files via web search — or from other documents/bibliographies — but they might later read the text in a Research Object context
- Executable Research Objects can embed PDF viewers tailored to their specific data sets

Which means ...

- Manuscripts may be cross-referenced with other Research Object content
- PDF Viewers could be programmed with extra code to implement cross-reference logic
- Page coordinates can be noted via `\pdfsavepos` and friends

Publications Within Research Objects

Individual text publications are still important because they are usually the place where readers learn about research work — e.g., either from a web search or referenced in a different publication. But once someone accesses a manuscript that is linked to a Research Object, they may then download that data set and conduct further reading and examination through the Research Object directly. If the original paper is included within that Research Object — in a format such as PDF — readers can view the manuscript via the included file rather than the web document they may have found initially. This creates new possibilities for annotations, cross-references, and interactive content, insofar as words, sentences, and other landmarks in article texts can be linked to data and code elsewhere in the Research Object.

What's "Cross-Reference Logic"?

Generic Examples

- Adding options to context menus in proximity of specific document contents
- Reading and acting upon hyperlinks (maybe something other than just scrolling to or loading the link target)
- Reading data within PDF annotations or comment boxes
- Adding GUI controls around page margins

Specific Examples

- If a graphic shows a video frame, play the video
- If video frames are geotagged, open a digital map display
- If a video records a simulation, open windows for the simulation graphics themselves, or to view initial parameters and/or implementation code
- Graphics specific to individual disciplines: e.g., chemical formulae may be linked with molecular-visualization files and windows
- Multimedia Cross-referencing: Consider a video showing an environmental remediation project, mitigating contamination from one or more toxic chemicals — potentially launching video, cartographic, and molecular display windows

What's "Cross-Reference Logic"?

Note that for a PDF rendering library, each PDF page is just a picture, which can be manipulated and overlaid with generic graphics functionality. The window coordinates of textual elements can be obtained from PDF annotations or by pairing PDF files with metadata compiled using techniques such as `\pdfsavepos` (from the LaTeX *pdfTex* package). This means that context menus activated over a PDF page can have different options depending on the contents of words, sentences, or annotations present near or around the mouse-event position. Also, the PDF display could be incorporated into GUI windows with additional buttons and controls. Whether via context menus, toolbars, or other controls, Research Object Applications can link PDF windows programmatically with other windows showing videos, digital maps, computer simulations, 3D graphics, and so on. Rendering engines such as Poppler or XPDF may be included directly in Research Objects, so PDF viewers could be customized for individual data sets and shared directly as source-code projects.

Text Mining

Helping researchers find publications relevant to their work — in theory

So Many Publications For most topics, too many peer-reviewed papers for researchers to consider each one individually. Can Text Mining and/or Natural Language Processing tools help?

Modeling Rhetoric The most useful papers for a researcher will typically not only be *about* some topic. They will, more specifically, *endorse*, *critique*, or neutrally *evaluate* an argument or theory

Thematic Connections If a paper cites another, what discursive role does this reference serve? To support a claim the current author makes or repeats from elsewhere; to indicate an influential precedent in the field; to identify an early example of where a theory or terminology was introduced; to direct readers toward a lengthier or more historically significant discussion of a topic; to single out the referenced work as exceptionally clear, influential, or innovative; or to initiate a critical analysis of the work?

Across Disciplines Computational tools might discover connections between subject-areas that researchers miss on their own

Text Mining

Alongside the emergence of open-access data sets and Research Objects, there has also been an increasing emphasis on text-mining and AI techniques to expedite academic research. The basic goal is to improve the accuracy of search results, by leveraging computationally tractable semantic cues that are more granular than simply searching for specific text strings — or even thematically related concepts. For example, there are thousands of peer-reviewed articles about Keynesian Economics, and merely identifying some publication as relevant to this overall topic does not clarify whether that document is important to some *particular* researcher's work about this topic. Instead, we'd like to discover more specific classifications, such as resources that *endorse*, *critique*, or neutrally *evaluate* Keynesian Economics; or analyze it from a macroeconomic perspective, or a sociopolitical one; or assess it differently from a moral versus quantitative angle, and so forth.

Case Study: "CORD-19"

- Over 400,000 full-text open-access papers about Covid-19, curated by Allen Institute for AI (AI2)
- Roughly 1/3 of the documents had JATS available; otherwise PDF extraction was required
- AI2 openly acknowledged the limitations of PDF extraction and issued a "Call to Action" encouraging publishers to develop better text encoding/metadata
- Converted all papers to S2ORC-JSON in the hope that developers could program text-mining tools facilitating COVID research

Case Study: "CORD-19"

The CORD-19 corpus, developed by the Allen Institute for Artificial Intelligence (AI2), is a good example of the goals and techniques for text-mining in the context of scientific research. This corpus includes over 400,000 freely-available full-text articles represented in both human-readable formats and in machine-readable encodings — specifically, S2ORC JSON (S2ORC comes from "Semantic Scholar Open Research Corpus"). Because Covid research spanned many disparate subject areas — viral morphology, infectivity, genetic profiles and strains, vaccine development, diagnostics, clinical treatments, epidemiology, etc. — automated text-mining tools could, at least in principle, help scientists discover research in other disciplines as they were trying fit the pandemic's pieces together. AI2 did not specifically create those tools, but they curated CORD-19 to be directly accessible from computer software in the hopes that other parties could build text-mining capabilities upon that basis.

Limitations of CORD-19

- Transcription errors for technical terms/formulae
- Documents were only modeled down to the paragraph level (not individual sentences)
- Limited support for semantic annotations and controlled vocabularies
- Inconsistent text encoding means that potential connections between disparate documents are not algorithmically recognized
- Machine-readable sources (XML, LaTeX, etc.) from publishers' internal workflows often unavailable

Limitations of CORD-19

According to AI2, roughly 30% of CORD-19's full-text documents were shared in structured formats such as JATS-XML, but the majority of publications were only available as PDF files. AI2 therefore had to extract text from those PDFs in order to perform conversion to S2ORC JSON. This is an intrinsically error-prone process due to anomalies within internal PDF text representation, such as hyphenation, ligatures, spurious space characters in some font styles, headers/footers and marginal content that needs to be isolated from the main text, etc. Furthermore, technical content such as chemical formulae or linguistics annotations can be difficult to read properly without working off of structured markup, such as XML. In general, transcription errors limit the extent to which inter-document connections and conceptual links can be properly recognized. Also, S2ORC only models documents at the paragraph level — not individual sentences — which can hinder the application of text mining techniques involving Natural Language Processing.

AI2 recognized these limitations and even issued a "call to action" encouraging publishers to develop more rigorous text encoding and metadata standards.

But Let's Not Get Too Locked In on Text Mining

There are other use cases for machine-readable text encoding

- Improving upon internal PDF search:
  - Load or embed JATS (or similar structures) alongside PDFs — then redirect PDF search to these files
  - Avoid problems due to ligatures, hyphenation, etc.
- More granular searching inside PDFs:
  - Restrict queries to footnotes, quotations, (sub)section titles, etc.
  - Correctly handle ellipses; quotation-emendations; citation data
  - Allow systematic use of semantic annotations
- Extend the scope of Full-Text Query Languages:
  - In conjunction with reverse-index (vector) databases (as current)
  - Searching "meta" indexes (pooling books' print/curated indexes)
  - Embedded in PDF Viewers or Executable Research Objects
  - As a standalone tool for authors and editors

But Let's Not Get Too Locked In on Text Mining

I think it is hard to judge the prospects of text mining as a research tool in these ways, sensitive to rhetorical structures and NLP nuance. Nonetheless, the emphasis on machine-readable text encoding — which has been spurred on by text-mining goals — is also important in other contexts.

Even without AI or NLP, rigorous text representation can improve upon native PDF search capabilities; help to implement text-to-data cross-references; and facilitate systematic, actionable semantic annotation schemes within published manuscripts. For instance, intra-document queries can be narrowed in scope to the main text; or, conversely, to block quotes, footnotes, section titles, etc. Moreover, special content such as chemical formulae, technical definitions, linguistics example sentences, proper names, GEO-taggable place names/locations, quotations, citations, and etc. can be parsed into structured data objects.

Sentence Objects

SENTENCES ARE THE BASIC UNITS OF DISCOURSE

- **Written Natural Language** XML, JSON, or other markup is merely an approximate serialization of textual and discursive structures
- **Sentence Nesting/Divergence Points** Sentences usually have a simple sequential organization, but not always (cf. footnotes, block quotes, enumerations)
- **Object-Oriented Implementations** Textual objects should be conceptualized and, as much as possible, exposed to computer code as class instances for Object-Oriented languages such as Java or C++. Markup should be intended for serialization, not to provide the definitive structure of documents for querying and other computational requirements.
- **Multiple Scales** Keyphrases and annotations offer a sub-sentence level of organization. Paragraphs and (sub)sections supply a super-sentence level.

Sentence Objects

There are numerous XML-based markup formats for text documents, including JATS, BioC, and TEI, plus non-XML options like S2ORC and TAGML. Any markup format, however, will encode natural-language text according to generic structures involving trees or graphs, rather than the inherent structure of written text itself. For instance, XML trees are defined in terms of parent and child nodes, siblings, elements' text content, and attribute values. Although there are analogous structures in natural language — for example, sentences in sequence are similar to sibling XML nodes — many textual phenomena can be modeled only partially via XML structure alone.

For these reasons, any markup file should be seen as the *serialization* of textual content, not as the definitive structure of that content. Conceptually, text elements are best understood as typed values in an Object-Oriented context. Moreover, I believe sentence objects — that scale of representation — should be the foundation of such object systems. Sentence objects are also contained inside larger elements (paragraphs, sections, subsections) and can encompass smaller parts (keywords, annotated fragments), but overall text-object systems should pivot around the sentence level.

Sentence Objects and Query Results

sentences.sdi (~/gits/pnp-dev/presentations/dev/

File Edit View Search Tools Documents Help

```
--- Abstract/start


--- Sentence/start
id: 0

--- Sentence//end
id: 0
r#  216  9  50
g.
|   @l(-%>) +19/6/1
|   +23/6/5:0=23/6/5
|   +33/6/15:0=33/6/15
|   +38/6/20:0=38/6/20
.   @l() -38/6/20
p: .
t.
|  In data sharing and publishing
|  contexts such as Executable Research Objects,
|  natural-language documents coexist with raw
.  data files and, potentially, multimedia resources


--- Sentence/switch
id: 1
r#  217  9  51


--- Sentence//end
id: 1
r#  318  11  53
p: .
```

sentences-result.txt (~/git

File Edit View Search Tools Documents Help

```
>find PDF :in $Sentences :save to %-result.txt
  :report-width 85
===



# 26  = PDF ->
|  Unfortunately, the most common format for d
|  is flawed from a computational perspective,
|  natural language often differs from how hum
|  (e.g., when seeing it rendered in a PDF vie

# 27  = PDF ->
|  Such details as ligatures, headers/footers,
|  artifacts in text extraction (or scraping)

# 28  = PDF ->
|  To clarify these remarks: PDF code librarie
|  to obtain text from one page and/or window

# 31  = PDF ->
|  The most straightforward contrast would be
|  occur, such as unexpected space characters
|  fonts (small spaces may be present in the i
|  readers, but methods like Page::text have n
|  characters)

# 33  = PDF ->
|  In addition to problems with ordinary langu
|  of special character strings such as organi
|  not use a consistent mechanism for notating
|  textual elements (or, at least, structured
```

Sentence Objects and Query Results

The prior slide shows examples of text which has been modeled at the sentence level. In this specific case, the text for the full-length article I am summarizing with these slides — which is programmed to generate both LaTeX and JATS files — also creates serialized versions of sentence objects. The resulting data files can then be parsed — in this case, into C++ objects — and those objects subsequently used for editing and text analysis. I have found this to be a useful system when preparing books and articles for publication. On the right, the prior slide displays how sentence objects can be employed for a demo full-text query system. The query can be observed as evaluated within the demo code published for this article. (This is not a full-fledged query language, but a provisional outline of the technology stack and implementation layers supporting such a language.)

Text Elements as Structured Objects

- Sentence objects themselves (the core of an effective object system for written language)
- Paragraphs
- PDF pages
- Proper names (different conventions of order, initials, alphabetization, middle versus compound)
- Place names (can be geotagged)
- Legal statutes/citations
- Sentence-like landmarks (definitions, statements of theorems/lemmas, axioms)
- Chemical formulae
- Citations (bibliographic reference, plus visible number or string, plus page range or other locator) — bidirectional links to bibliography
- Quotations (need to model ellipses, emendations, paragraph breaks, sources) — show source in separate window?
- Index locators (visible Para ids? Mark content matching index headings?)
- Para id subdivision (e.g., inside/around block quotes)
- Figures, images, and other graphics content — possibly interactive

Text Elements as Structured Objects

I use the term "Sentence Object Model" to describe text encoding built around sentence objects — although objects can encompass a spectrum of types both at higher and lower scales. Among sentence objects themselves, there is usually a straightforward structure — where sentences follow one another in sequence, belonging to encompassing paragraphs — but sometimes the structure is more complex. For instance, footnotes may contain multiple sentences or even paragraphs, but they are in some sense children of the sentences where the mark appears — or, if they come immediately after a sentence, the sequence of sentences effectively diverges into main text and footnote content. Similarly, a block quote or enumerated list often functions like a child of the sentence that introduces it. Sometimes that preceding sentence is grammatically incomplete and has the effect of merging with the content that follows it. Also, special content such as quotations, citations, bibliographic entries, and entries/locators for print indexes have their own structural requirements. As these examples suggest, modeling the full organization of textual content requires an ability to represent sometimes complex objects and their interrelationships.

Para Ids and Sentence Boundaries

Grammar overall, in terms of "Structural Empiricism":

i

> Viewed through the lens of structural empiricism, Cognitive Grammar is therefore a family of models of a linguistic system. Like any other scientific theory, it features theoretical entities and processes (cognitive domains, constructional schemas, compositional paths, subjectification, etc.) and empirical substructures isomorphic (in a weaker version: similar) to appearances of observable phenomena.[27, p. 18]

a

He immediately avows "It is somewhat unclear what should count as observable phenomena in linguistics", but situates such uncertainty in a larger philosophical context. In particular, *most* scientific theories entertain taxonomies and analyses of a domain of entities encompassing both public observables and "hypothesized" posits that — for one of several possible reasons — cannot be "seen". Empiricism is *structural* insofar as such "unobservables" are taken seriously because they contribute to persuasive explanations of *observables'* behavior, with structured models unifying the seen and the hidden into a rational package. Sometimes unseen posits are deemed just as real as everything else, merely beyond our capabilities of (instrument-enhanced) observation, at least for now. Other phenomena (like "dark energy") are left more open-ended, possibly subsumed under such ontological realism or possibly artifacts of some more complex process (e.g., "phonons", which behave like "particles" of sound, but overlooks the fact that many useful scientific models involve clearly counterfactual scenarios [whose] role is not merely heuristic or pedagogical.[27, p. 32]

He then analyzes how

a

ii

> fictional models in natural sciences can be accounted for in philosophical terms. In ... structural empiricism, isolated systems, frictionless pendulums, and perfect liquids are treated just like any other theoretical objects, except their theoreticity does not follow from unobservability, as is the case with electrons and electromagnetic fields, but from their fictional character. [A] structural empiricist is not committed to believing in the actual existence of real-world objects corresponding to theoretical concepts, but only to the empirical adequacy of the theory featuring these concepts.[27, p. 33]

A reasonable paraphrase vis-à-vis constructed sentences is that, for purposes of expositing semantic or syntactic claims, the universe of linguistic objects is broader than merely the totality of sentences humans have in fact created. Ad-hoc hypotheticals also belong to that domain if they serve and fit within model-building regimes.

b

I would put it thus: what makes a sentence an object in the domain of entities spotlighted by linguistics is not the fact of its being empirically an example of real language-use, but rather how upon that sentence we can apply ob-

Para Ids and Sentence Boundaries

The prior slide and the next show a sample document that illustrates one way to leverage paragraph subdivisions and explicit sentence boundaries. Publishers have started to use paragraph ids instead of page numbers to record index locators during manuscript preparation. The obvious benefit of para-id's is that they remain constant even while pages get renumbered due to changes elsewhere in the document. They are also more granular than page numbers alone, directing us to specific parts of a page — especially in conjunction with 2-column text.

Ordinarily, para-ids are replaced by page numbers in the final publication and usually are not visible to human readers. However, it is possible that future publishers will choose to keep para-ids as marginal content, to help human readers locate and refer to specific locations on a page. Visible para-ids also introduce hyperlink sources and targets to support navigation back and forth between an index and the main text.

Continuation Marks: a, b, c; i, ii, iii; left/right column

just as real as everything else, merely beyond our capabilities of (instrument-enhanced) observation, at least for now. ⇉Other phenomena (like "dark energy") are left more open-ended, possibly subsumed under such ontological realism or possibly artifacts of some more complex process (e.g., "phonons", which behave like "particles" of sound, but that's largely a metaphor [33]).⇉Natural selection or "design" is not an object, but rather an encompassing system wherein adaptation drives evolution.⇉Potentially "dark" matter and energy will ultimately be given a similarly indirect and holistic cosmological interpretation, rather than deemed to refer to a specific kind of field or particle.↴

...45 is well-posed.⇉Even *imagined* speech is grammatical (or not).⇉We can label syntactic categories; track pronoun-antecedent pairings and singular or plural alignment; observe how inflections indicate verb-tense; and etc.: minimal operations in the syntactic and semantic realms that apply equally whether samples are real or hypothetical.↴

46 ⊱My earlier subway-platform speculation was similarly about hypothetical circumstances.⇉I have caught New York subways in real life, but that's beside the point: we are primed to learn from experience, but in a manner

aled.⇉Ad-hoc hypotheticals also belong to that domain if they serve and fit within model-building regimes.↴

⊱I would put it thus: what makes a sentence an object in the domain of entities spotlighted by linguistics is not the fact of its being empirically an example of real language-use, but rather how upon that sentence we can apply observations, analyses, annotations, etc., whose ordinary ground is real-world speech-events.⇉We can, for instance, note syntactic, morphological, or lexical patterns, and point out where pragmatic interpretation is needed.⇉Constructed sentences may or may not conform to English grammar, but at least the question of whether they do so

45

two verbs.⇉The sentence is not merely notating observable details: choice of words can encode information about the speaker's epistemic starting-point.⇉This phenomenon is nicely illustrated in (15) and (16), constructed or not.↴

⊱But let's step back.⇉For sake of discussion, I will assume (15) and (16) occur in contexts where listeners know the speaker saw John firsthand (the evidence is direct observation, not hearsay).⇉Implicit in a communicative claim to *describe* something witnessed is having been in a perceptual orientation to *see* it.⇉The sentences then encode

48a

49

Continuation Marks: a, b, c; i, ii, iii; left/right column

With respect to para-ids, one nontrivial question is how to define the scope of individual paragraphs. Many paragraphs in a typical document have identifiable smaller parts — for example, we might have the start of a paragraph, then a block quote, followed by further discussion, then perhaps another block quote. This sample document uses a sub-indexing scheme to identify sequential parts of a paragraph within the main text (via letters **a**, **b**, **c** etc.) and block quotes inside paragraphs (via roman *i*, *ii*, etc.). Moreover, special marks identify portions of paragraphs that run onto a new page or the top of a page's right-hand column.

This graphic also shows special marks added to identify sentence boundaries — or, more precisely, the boundaries that have been modeled via metadata to delineate sentences. Such marks ordinarily would not be visible to typical readers, but they could be used by authors, editors, or programmers to double-check how sentence boundaries are computationally recognized.

Paragraph Subdivisions and Index Locators

Sentence Examples ⋆

[1] My brother has been married for 10 years, but in many ways he's still a bachelor. 1 (2)

[2] All of the eligible bachelors in this town are married. 1 (2)

[3] Everyone sang along to three songs. 1 (3)

[4] All appeals are heard by a panel of three judges. 1 (3)

[5] All New Yorkers live in one of five boroughs. 1 (3)

[6] All New Yorkers complain about how long it takes to commute to New York City. 1 (3)

[7] In Ariel's picture of Beatrice, she looks sick. 1 (3)

[8] In Ariel's picture of Beatrice, she snapped the lens just as two cardinals flew by. 1 (3)

[9] We operated near the bank. 1 (3)

[10] On the mantle was a wooden bat. 1 (3)

[11] Every band performed three songs. 2 (7)

[12] Everyone sang along to some song or another. 2 (7)

[13] This election is slipping away from the Democrats. 2 (8)

[14] Student after student came by to complain about the reading assignments. 2 (8)

[15] John poured water onto the stove. 8 (48)

[16] John spilled water onto the stove. 8 (48)

[17] I put Mom's chocolate brownies in this red tin box, take some! 9 (60)

[18] I'm heading out to the store. 13 (80)

Index †

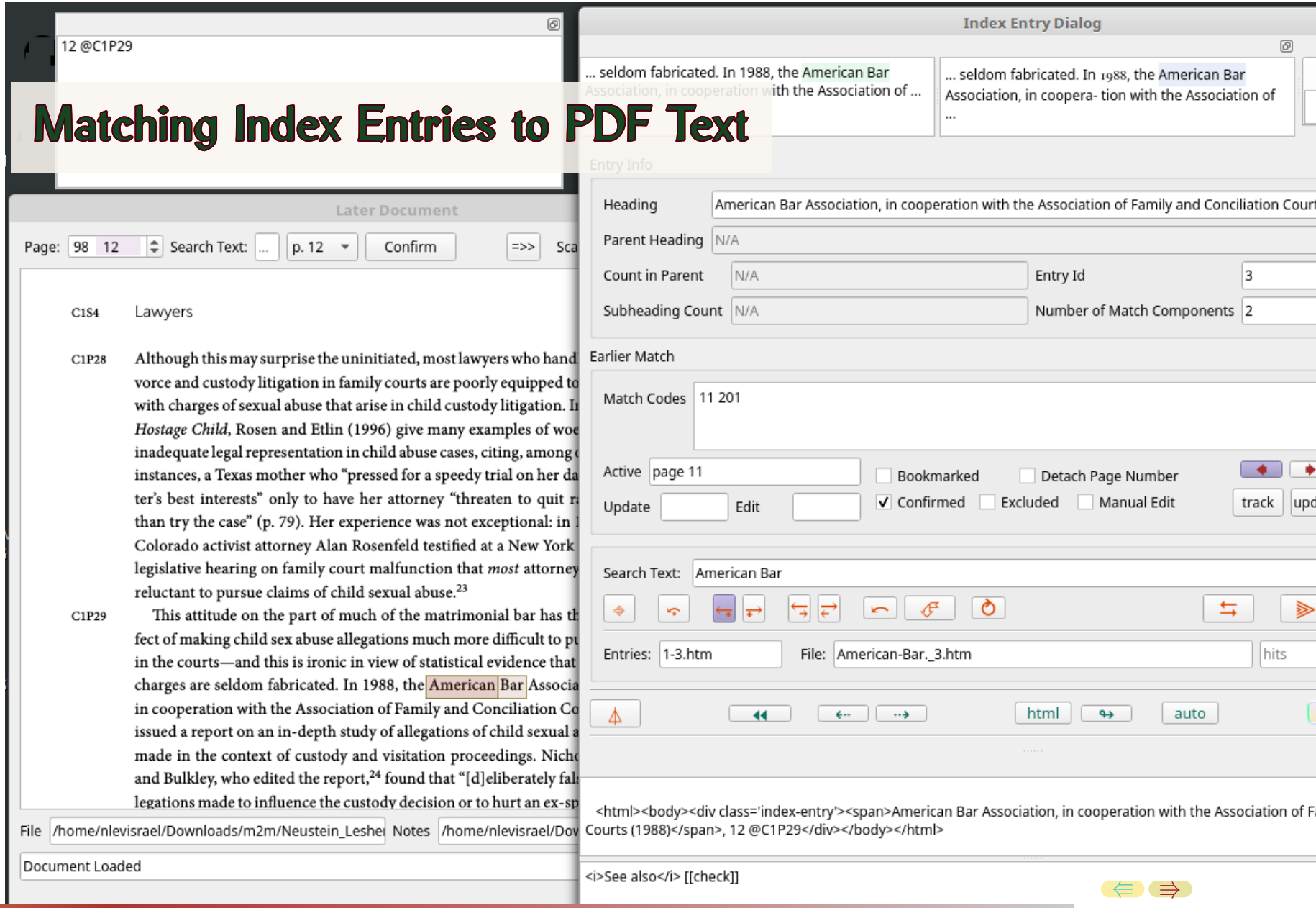
6 (36) (38)

Paragraph Subdivisions and Index Locators

The prior slide shows the index of the same document. Ordinarily articles do not have a print index — only books — but that convention too might change as text encoding becomes more rigorous. Here, the index locators are provided both via page numbers and via para-ids with subdivisions. Clicking on page numbers scrolls to the top of the relevant page, whereas clicking on a para-id brings up the corresponding page just above the desired paragraph (or subdivision thereof).

On the left-hand side of this page is a list of example sentences discussed in the paper — a common pattern for linguistics articles. I believe it should be adopted as a conventional practice in this discipline to have all such samples reproduced at the end of a document, similar to a glossary. Moreover, example sentences can be exported as metadata and merged into linguistic corpora. Similar techniques could be used for other academic fields wherein publications have special kinds of segments or environments that may be coalesced into glossary-like structures.

GUI Controls for (Print) Indexing

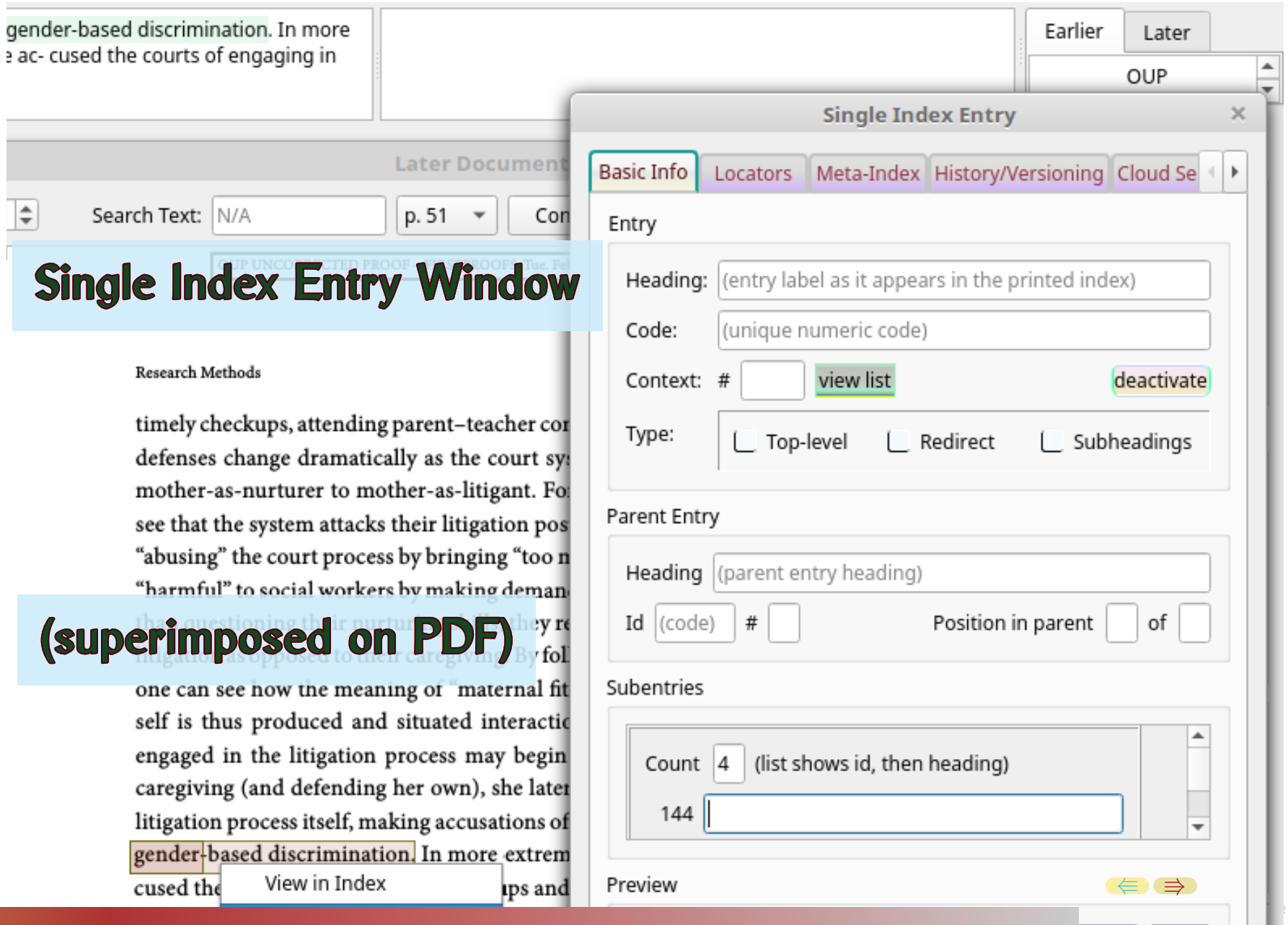


GUI Controls for (Print) Indexing

Once text is compiled into sentence objects — represented, for example, by C++ classes — then the C++ data may be queried to support document-preparation tasks, including editing, checking references, and compiling an index. Ideally, queries are routed both to internal PDF searches and to text/element objects.

"Indexing" in this context does not refer to construction of a web index — which in most cases is technically a "reverse" index, mapping key words to the set of documents that contain them (or contain semantically similar words). Reverse indexes are typically machine-generated and provide only an approximate model of documents' content based on word- or concept-occurrence vectors. A print index, by contrast, is manually compiled and reflects the authors' domain-expert understanding of which terms deserve separate index entries, and which paragraphs are relevant matches for those concepts.

GUI Window for Single Index Entry



Single Index Entry Window

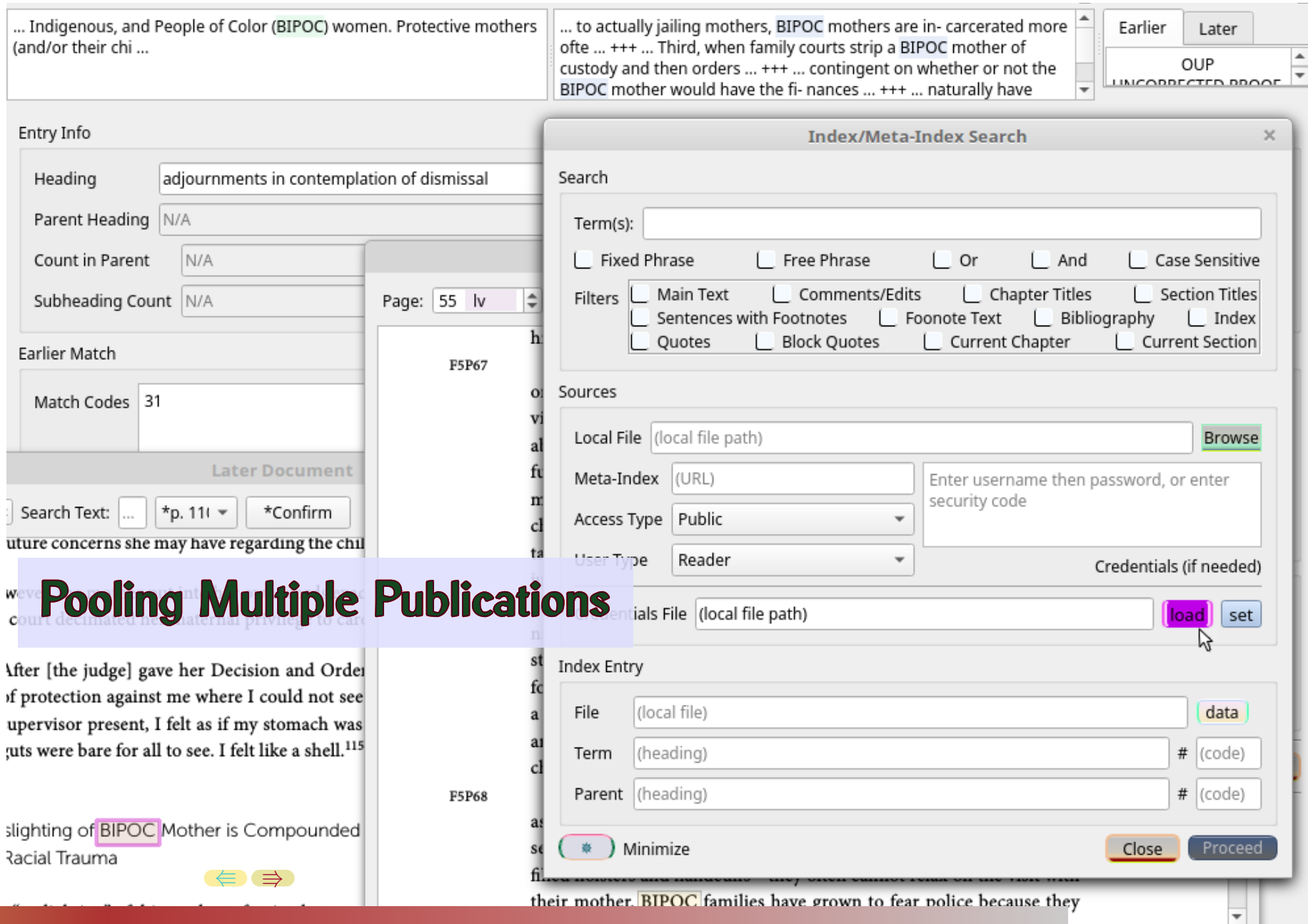
(superimposed on PDF)

GUI Window for Single Index Entry

Even one single index entry can be relatively complex, with multiple locators — that is, multiple pages or paragraphs notated as a conceptual match — plus potential "*see*" and "*see also*" redirection, and/or potential subentries. During document prep, it is entirely possible to implement special-purpose GUI windows specifically dedicated to individual index entries.

For any given project, it may be an open question whether the development effort needed for general-purpose indexing GUIs is worth the added convenience, because some users might actually find it easier to work with text files serializing lists of index entries. Nonetheless, a reusable suite of indexing GUI tools could certainly be part of a comprehensive software package for Executable Research Objects.

Meta-Index Search



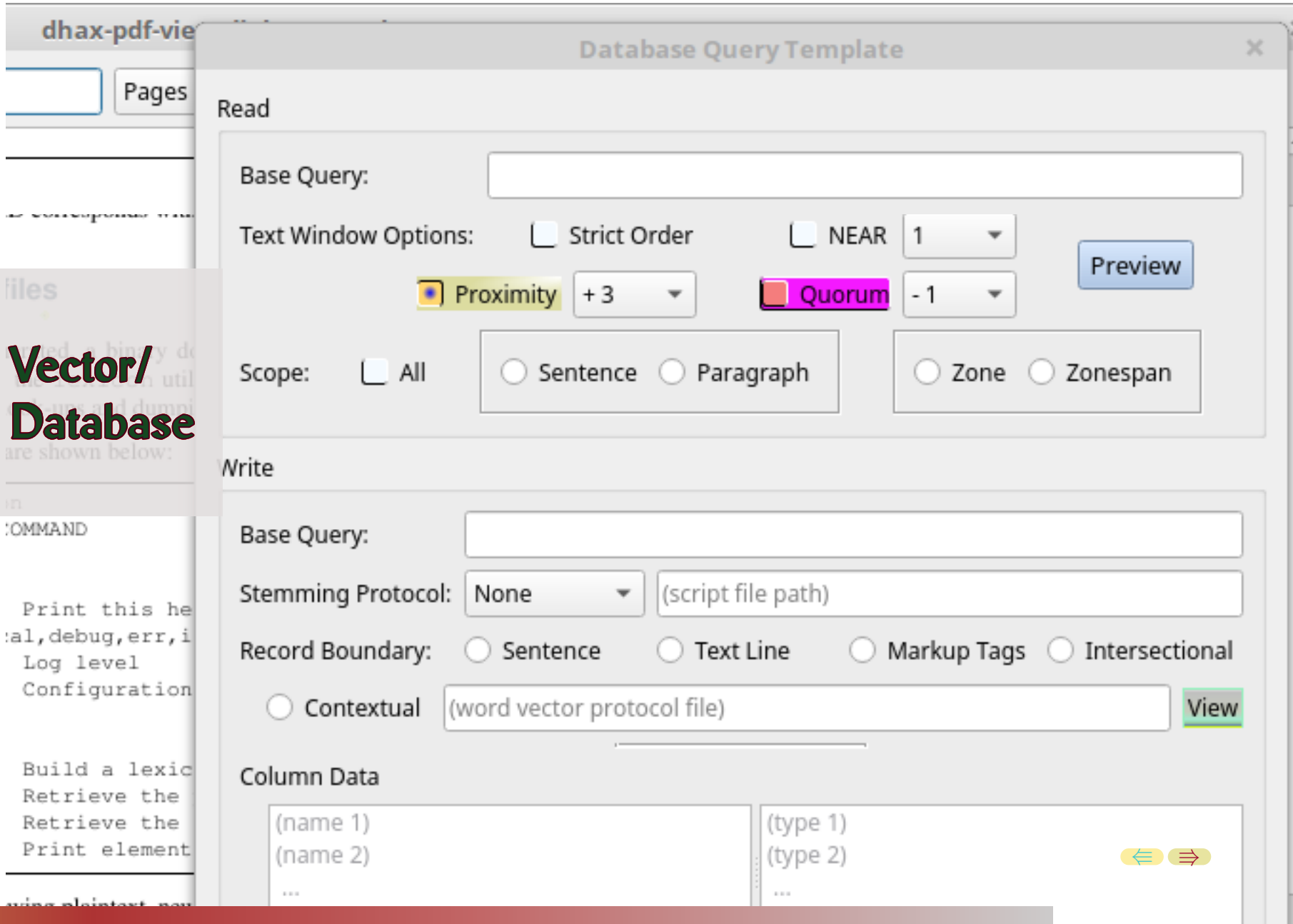
Slides accompanying "Merging Full-Text Query with Data Sets: A perspective from compiler theory" by Nathaniel Christen

Meta-Index Search

Once authors or editors have expended the effort to create print indexes for individual publications, a natural extension of that labor would be to pool indexes of related documents, such as those published within a single journal or book series. Meta-index web services could then be accessed from individual publications via PDF extensions (as well as accessed through ordinary web portals).

Meta-Index searching can be contrasted with — and performed alongside — broader reverse-index searches. On the one hand, meta-indexes are narrower in scope because they presumably apply to smaller document collections. This would be particularly true if meta-indexes were combined with full-text representations that could be searched as rigorously as local files, without the simplification and data-compression of reverse indexes (which use word stemming/lemmatization and generally discard sentence contexts). On the other hand, meta-index searches can be more granular and contextually fine-tuned.

Reverse-Index Searches

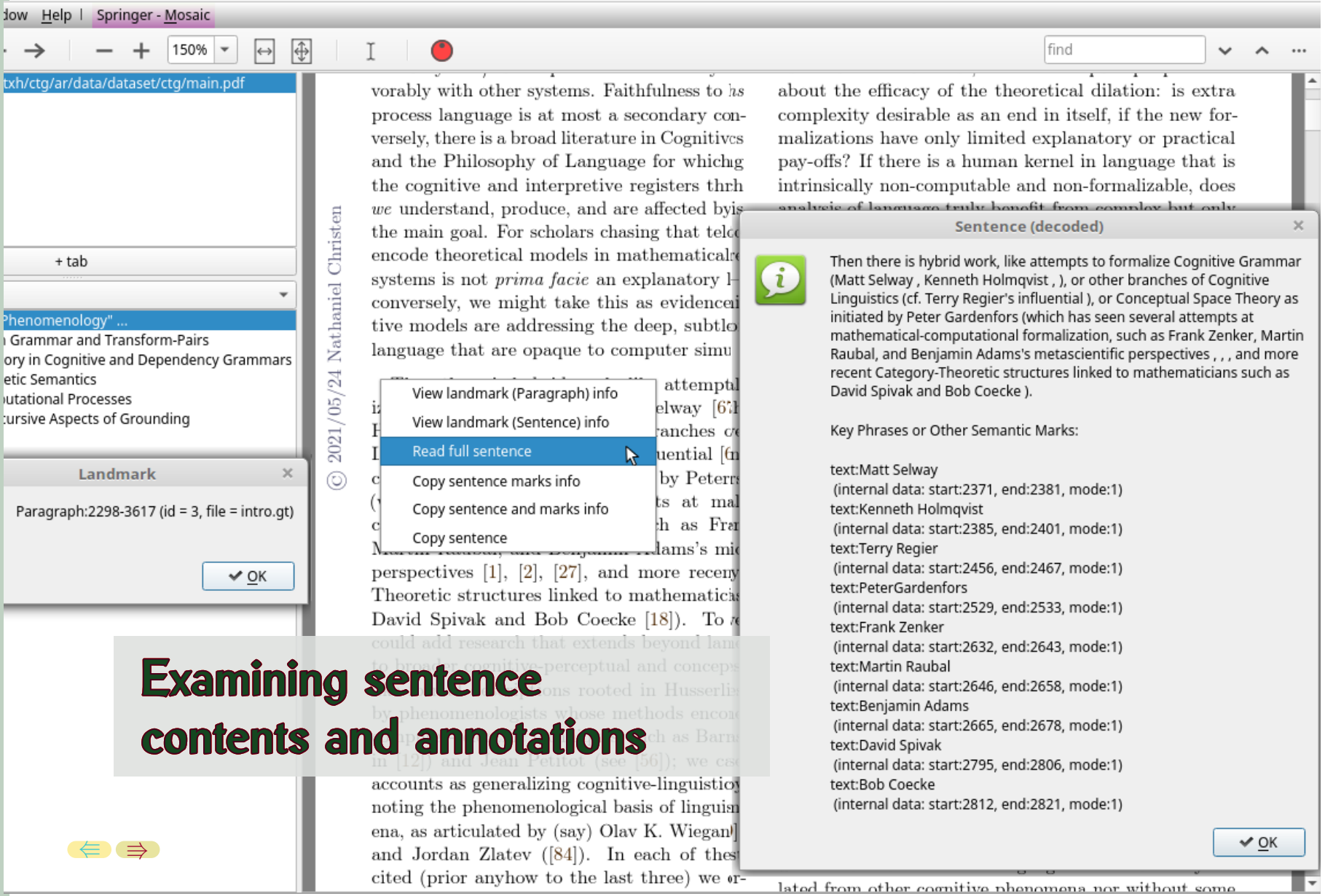


Reverse-Index Searches

In contrast to meta-indexes, the reverse-index lookup employed by search engines is a much older technology. Nonetheless, it might be useful to design GUI windows specifically organized to initiate queries against conventional full-text search engines.

Full-text queries often have multiple input parameters that serve to adjust how the search operates. Usually these details are invisible to end-users, but on some occasions users might want to exercise fine-grained control over search policies. This might be achieved via HTML forms in the context of web services, but dedicated GUI windows could be designed instead in the context of native-compiled document-prep tools. For example, these searches might help authors or editors find relevant publications for citations and bibliographies.

Viewing Sentence and Annotation Boundaries



Examining sentence contents and annotations

Viewing Sentence and Annotation Boundaries

Representing text at the sentence level allows queries to be evaluated that focus on individual sentences as containers and/or the basis for reporting query results. For example, queries might search for words or phrases with the stipulation that they appear specifically at the beginning of a sentence, or all in the same sentence. As another example, one could locate the first sentence in a document, or in a chapter, where the search text appears.

The prior slide shows functionality that could be enabled when sentence-object data is read by code within a PDF viewer. Here, the user has activated a context menu to read the contents of the current sentence (a related context option would be to copy the current sentence to the clipboard). The resulting dialog window also shows the locations of proper names and annotations within the current sentence.

Data Objects for Executable Research Objects

So, Research Objects' internal information space covers two main kinds of objects (in the sense of class-type values)

Document-related Objects

- Sentence objects and other text elements
- Objects related to manuscript versions, changes, provenance
- PDF annotations and comment boxes
- Index entries and locators (headings, subentries, "*see*" and "*see also*" redirection)

Dataset Objects

- Individual records/observations as class instances
- Collections of records/observations (tables, lists, associative arrays, queues)
- Individual fields/measurements as class members
- Objects encapsulating structured data such as MIBBI protocols
- Multimedia assets and GUI controls that render them

Data Objects for Executable Research Objects

So thus far I have talked about an Object-Oriented model for sentences and other textual elements, and also about Research Objects and data sets. We can assume that, in many cases, Executable Research Object code will be working with both kinds of data — that is, textual objects obtained from sentence-based encoding and data objects deserialized from raw data files. The Research Object code will need capabilities to navigate, examine, and search against collections of objects whose class types cover both text and research contents. Merging these forms of data into a common information space affects several dimensions of Research Object implementation, including searching, coding practices, documentation, and GUI design. We can consider how search and semantic indexing techniques might be extended to data types, fields, and procedures implemented by dataset code.

Cross-References (again)

Cross-referencing between Text Objects and Data Objects

- Column names, units of measurements, scales, etc., can be matched to text strings
- PDF page areas can be given interactive capabilities to launch GUI controls oriented toward specific object classes in a data set
- Text/Code cross-references: data types, procedures, preconditions, source files
- Algorithms and analyses: computational processes discussed in the text and implemented in Research Object code

Cross-References (again)

With respect to searching, in numerous contexts the same semantic models that apply to natural language carry over to structured data objects, at least in some facets. For example, any data structure that can be arranged into tables has a notion of column labels, and their terminology may well match phrases present in associated publications. Measurable quantities are often expressed in units of magnitude, which are also semantic entities — as an example, we might want to search for every field in a data set expressed in latitude/longitude coordinates. Code procedures that accept arguments dimensioned according to specific units of measure are also semantic matches for those units. In the case of GIS, for instance, algorithms that convert to or from other coordinate systems — such as "XYZ" — would be reasonable matches for terms "latitude" and "longitude". In general, semantic indexing can be extended to data objects and computer code.

GUI Indexing for Research Objects

- Context Menus: each menu has a list of options, including label text and actions/callbacks
- Drop-Down Lists or "Combo" boxes: often items can be associated with a controlled vocabulary
- Mouse gestures: where different gestures are possible and what they mean ◆ Drag ◆ Key-press modifiers ◆ Cursor icons
- Separate windows: top-level, dialog, wizard
- Model-View Controller pattern: each GUI class is correlated with a data class — component's state at any one time shows one object
- User manuals ◆ Interactive documentation (as a special GUI mode) ◆ Functionality (remember, windows can get reorganized)
- Key-combo extensions — bind keypress combinations to mini-scrips (*cf.* Emacs LISP)

GUI Indexing for Research Objects

Semantic indexing can also be extended to GUI design. Some examples include the following: any context menu has a list of options, which might be determined by the local object where the menu was activated, and which are all semantic entities. The same is true for controls wherein a user may select one of multiple options, such as QComboBox — each instance is effectively a controlled vocabulary. Many GUI classes have "actions", which are methods that can be explicitly initiated by users — i.e., they represent the handlers for class functionality directly available to users via menus, buttons, and other controls. Another facet relevant for indexing is the different kind of user gestures applicable for a given GUI object, including button-press, dragging, and turning the scroll wheel.

Ideally, Executable Research Objects — like any GUI application — will have a manual or documentation helping users to find specific units of functionality. For instance, any GUI window could be catalogued with a list of controls inside it, their purpose, and how users can interact with them (by clicking, dragging, etc.)

Computational Support for Text/Data Integration

Scripting and Query Languages

- Full text query built around "Sentence Objects"
  - Query results can take the form of sentence lists
  - Sentences provide a match context (e.g., find two words in the same sentence)
  - Restrict matches to the beginning of a sentence, or the interior
  - Find the first sentence in a chapter (or section, etc.) where a match occurs
  - Find annotated text-to-data cross-reference points, in PDFs and text encoding
- Scripting for Research Object data
  - Deserialize raw data files
  - Customize GUIs for individual data sets
  - Implement algorithms, analyses, data types
  - Automate unit tests
  - Define workflows, simulations, data-processing steps or protocols
  - Provide standalone applications as Executable Research Object entry points
  - Customize PDF viewers for individual Executable Research Objects

Computational Support for Text/Data Integration

Assuming that one has now merged research text and data into a common object system, we can then explore how to benefit from the resulting, unified information space. If we envision that an Executable Research Object is implemented in a language like C++, then provisional access to the object system would come via C++ code. However, scripting and/or query languages offer a different interface to the underlying functionality with some benefits of their own: allowing C++ classes to be reused by multiple data sets (because script code could implement the finer details for individual projects); for unit testing and documentation (scripts can generate "user manuals" instead of relying on source comments); for implementing text queries by authors or editors without having to write and compile new C++ code. (As one example, "manual-generating" scripts could apply code-introspection techniques together with LaTeX or XML templates.)

Using Scripts to Enhance Research Object Presentations

- Avoid relying on external deserialization libraries
- Design by Contract (DbC): Explicit preconditions and programming assumptions
- Explicit scales, coordinate systems, units of measurement
- Use dataset code to clarify theoretical background, data acquisition protocols, data models, and formulae/subroutines documented via procedure implementations

Using Scripts to Enhance Research Object Presentations

Supposing an Executable Research Object is primarily coded in C++, we can still use a scripting language to program specific capabilities. For instance, a language could be chosen (or implemented) which is self-documenting and emphasizes explicit interface design more aggressively than C++. Design-by-Contract (DbC) techniques may be employed to explicitly notate procedures' assumptions and preconditions. Units of measurement and coordinate systems can be inherent parts of the type system — e.g., a pair with elements labeled as $\{x, y\}$ distinguished from {row, column} or {width, height}. Algorithms or formulae that might be described cryptically in article text can be presented in a more accessible manner within computer code. In general, code libraries can help demonstrate the theories and protocols affecting research work — especially if programming languages are designed to prioritize this pedagogical dimension.

The Trouble with Deserialization

Generic or Custom Formats

Some discipline-specific data-encoding formats are based on generic languages such as XML or JSON. Others use domain-specific markup or representation formats. .

Navigation and Traversal

For those based on generic formats like XML, deserialization code must still navigate through a DOM tree in order to extract the specific data fields needed in each context.

Custom Parsers

For discipline-specific representations, an additional step is input files to create tree or graph structures to begin with.

Library Maintenance

In either case, deserializing data encoded to domain-specific standards requires dedicated computer code. Consequently, individual organizations and academic departments often take responsibility for maintaining libraries geared to deserializing specific data formats, typically those associated with their particular research specialization.

The Trouble with Deserialization

Focusing on deserialization in particular, there are many domain-specific data formats such as FASTQ and VCF for genomics; HDF5 and ROOT for physics; FITS and netCDF (astronomy and earth sciences); PDB, MOL, SBML, and CELLML (biochemistry); IFC and BIM (architecture/engineering); SEGY and BAG (geophysics and oceanography); CoNLLU (computational linguistics); DICOM and OME-TIFF (bioimaging); and GIS-indexed attribute layers and shapefiles for geospatial data sets. Whether or not these formats are based on generic standards like XML or JSON, or alternatively need their own special-purpose parsers, it usually requires special code to convert raw data files into usable data objects. It is obviously impractical for everyone to write such code from scratch simply to access raw data files — which is why organizations often provide deserialization libraries for common use. But such libraries can have limitations, including lags in updating algorithms for new format specifications and difficulties in porting code to different computing environments. In this case, I believe a scripting language built with a lightweight and self-contained technology stack could alleviate some of these problems.

Language Stacks for Research Objects

Compiler and Virtual Machine architecture for Executable Research Objects

Support both query and scripting languages There are use cases for both forms of computer language in Research Object contexts

Minimize External Dependencies As much as possible, language compilers and runtimes should be built directly from source files internal to data sets

Customizable It should not take extensive extra coding to add syntactic and runtime features designed specifically for individual data sets

Focus Language Design around Research and FAIR Sharing Special languages inside data sets are not intended to be general-purpose scripting languages. Instead, their syntax and semantics should be focused on the goal of documenting research methods, theories, and data models

Provide Compiler and VM Support for Desired Language Features Design by Contract, unit-decorated types, coordinate-system-specific types, multiple-dispatch, MVC architecture, and other patterns for semantically detailed languages require bottom-up support vis-à-vis instruction sets and runtime state

Language Stacks for Research Objects

So these are some use cases for a scripting language specific to Executable Research Objects — and I will also discuss full-text search and how something like this could also serve as a query language. But just to talk about implementation: the idea here is specifically that a language be built *inside* a Research Object, with all the code layers published internally as source files — minimizing external dependencies as much as possible. Certainly we should not rely on heavy, complex dependencies such as LLVM or libClang. The tech stack should be built from the bottom up with an emphasis on functionality specific to Research Objects: deserialization; • Design by Contract as a pedagogical as well as design tool; • documenting scientific theories and data models; • generating manuals and interactive tutorials; • text/data integration. And the language should be customizable, with the option of adding new grammar rules, VM instructions, and built-in runtime functions for each data set.

Parsing for an Embedded Compiler

Useful design patterns for parsers oriented to self-contained compiler stacks

Avoid relying on external tools such as Lex, Yacc, Bison, LLVM, etc.

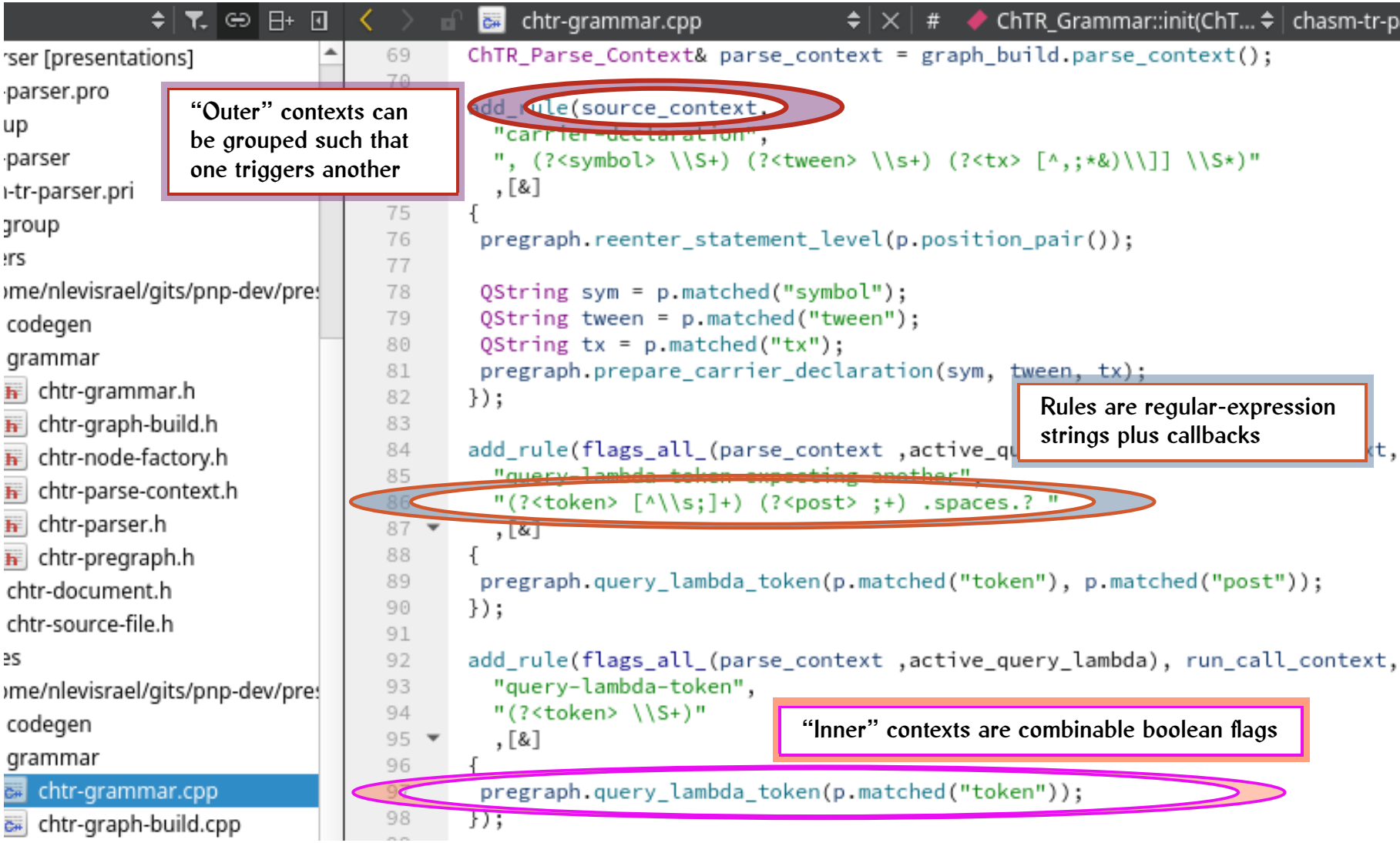
- Grammars should be context-sensitive: they might have one or more notions of "context" to support language extensions and expressive syntax
- Individual rules are regular expression objects coupled with callback handlers, and optionally contextual filters: we are not generating code *for* a parser, we are just implement rule-lists that are tried in sequence — anchored at the current "cursor" position — until one regex matches
- Rule callbacks can modify parser state/contexts, but in general would emit code for a "first-stage" Virtual Machine
- This preliminary VM then calls instructions for a code generator that produces "bytecode" for the main VM
- In both contexts "bytecode" is actually calls to class methods, and can be printed in human-readable fashion (or compressed to opcodes for production)

Parsing for an Embedded Compiler

Conceptually, a compiler pipeline begins by reading input files into a "parse graph". In a schematic sense, each grammar rule presents an opportunity to add a node to the graph. Rules can be checked sequentially at the current "cursor" position — activated or deactivated via contexts — with the first matching rule preempting others (so that rule-order is a structural feature of the grammar). To avoid reliance on external parser-generator tools, each rule can actually be a normal regular-expression string coupled with a callback handler (e.g., anonymous/"lambda" functions). When the graph is complete, it may be subject to transformations — for instance, macro expansion — and then, in its final form, is traversed strategically to generate VM bytecode.

This is a useful outline, but the workflow in practice may be a little different. I have found it expedient to implement a "preliminary" VM whose role is to assemble the parse-graph and/or generate bytecode — in other words, rule callbacks do not create nodes on their own, but rather emit "provisional" VM code. And, in the system discussed here, "bytecode" is actually just methods of specific C++ classes, but designed to emulate things like opcodes, registers, and stack frames.

Rules and Contexts in Parsing Code



Rules and Contexts in Parsing Code

The prior slide shows a small segment of the grammar from demo code I have published alongside this presentation. I want to call attention specifically to the notion of context, which actually occurs here on two levels. Although the rule definitions are based on regular expressions, they are actually preprocessed — allowing a slightly expanded regex format — and then initialized into rule objects that test current contexts. "Outer" contexts serve more general purposes and can be activated together — i.e., one context can trigger others when it comes into effect. For instance, outer contexts might distinguish ordinary code from extended, structured comments for documentation. "Inner" contexts are boolean flags and usually active over a shorter span. For example, an inner context might be rules active only inside quote literals. In this depicted code, the inner context goes into effect within the interior of statements parsed specifically as full-text queries (a special input channel).

The Idea of "Channels"

- Group together input and/or output parameters with related (maybe special-purpose) semantic profiles
- Language extensions can be implemented via special types of channels (e.g., supporting *queries* inside *scripting* formats by building channels that model query parameters)
- Back-door channels can implement proof-checks and other DbC features
- Channels provide convenient search-points for static analysis or runtime monitors

The Idea of "Channels"

Another structure I find it useful to introduce into script-compilers is what might be called "channels". Conceptually, functions in computer code — not too different from mathematics — have zero or more inputs and return zero or more outputs; i.e., there is one tuple for input parameters and one for output results (though for many languages we can in fact return just one value, at most). However, even mainstream languages complicate this picture via "**this**" objects, for instance, that are notated apart from other inputs and may even be passed invisibly; and via exceptions, which are detached from typical return-value semantics. One might envision these constructions as distinct input and output channels — and we can generalize this observation: languages could employ special channels to represent arguments that have some semantic or syntactic separation from ordinary parameters and therefore require special compiler or runtime treatment. Implementing a channel system allows these special code-points to be identified and isolated, for static analysis or runtime monitoring. Channels permit flexible, "user-defined" calling conventions.

Building Channels Programmatically or From VM Code

```
@fn --sf-- ;.

init-source-file-lexical-scope ;.
source-file-index $ 1 ;.

 .; statement-level declaration ;.
load-type-u1 ;.
declare-lexical-typed-symbol $ x

 .; statement-level pin ;.
single-init-pin $ x ;.

load-value-literal $ 5 ;.
resolve-pins ;.

 .; statement ;.
statement-line-number $ 5 ;.
init-new-ghost-scope ;.
push-carrier-deque ;.

new-call-package ;.
add-new-channel $ proc ;.
load-proc-name $ prn ;.

 .; expression ;.
statement-line-number $ 5 ;.
push-carrier-deque ;.

new-call-package ;.
gen-return-channels ;.
add-new-channel $ proc ;.
load-proc-name $ + ;.

add-new-channel $ lambda ;.
load-carrier-symbol-lxs $ x ;.
load-unsigned-literal-int $ 6 ;.
```

chasm-lib-console/main.cpp* # testqs1(QString, fl...

```
188
189 void run_testqs1(Chasm_Runtime* csr)
190 {
191  Chasm_Call_Package* ccp = csr->new_call_package();
192  ccp->add_new_channel("lambda");
193
194  QString a1 = "A1";
195  float a2 = 12345.6789;
196  u1 a3 = 135;
197
198  Chasm_Carrier cc1 = csr->gen_carrier<QString>(&a1);
199  Chasm_Carrier cc2 = csr->gen_carrier<r8>(&a2);
200  Chasm_Carrier cc3 = csr->gen_carrier<u1>(&a3);
201
202  ccp->add_carriers({cc1,cc2,cc3});
203
204  ccp->add_new_channel("retv");
205  Chasm_Carrier cc0 = csr->gen_carrier<u1>(csr->Retvalue._u1
206  ccp->add_carrier(cc0);
207
208  csr->evaluate(ccp, 301361_cfc, &testqs1, &cc0);
209  u1 result = cc0.value<u1>();
210  qDebug() << "r = " << result;
211 }
212
213 void run_testqs1n(Chasm_Runtime* csr)
214 {
215  Chasm_Call_Package* ccp = csr->new_call_package();
216  ccp->add_new_channel("lambda");
217
218  QString a1 = "Test";
219  u4 a2 = 33;
220  u1 a3 = 111;
221
222  Chasm_Carrier cc1 = csr->gen_carrier<QString>(&a1);
223  Chasm_Carrier cc2 = csr->gen_carrier<u4>(&a2);
224  Chasm_Carrier cc3 = csr->gen_carrier<u1>(&a3);
```

Building Channels Programmatically or From VM Code

In the demo code, channels are built up incrementally by specific C++ classes. A channel "package" then serves as the container object through which precompiled C++ code (as well as procedures defined in script) may be invoked via function pointers. In order to test the channel aggregation and/or foreign-function interface, it is possible to simply call the relevant C++ methods in sequence, from other C++ code — an example of this approach is visible on the right on the previous slide. More commonly, of course, the steps to build up channel packages are exposed as VM instructions. An example of VM code designed for this kind of system — again, taken from the paper's demo repository — is overlaid on the left.

Hidden or "Back Door" Channels

Real and hypothetical examples

- The implicit "message receiver" or "*this*" object — compilers will match procedure names to class methods even if *this* is not explicitly present among arguments
- Memory buffer to be initialized via Return Value Optimization
- "Proof" objects to prevent functions with Design by Contract stipulations from being called unless preconditions are known to have been tested
- Symbols captured by a closure
- Exceptions propagated outside a *try/catch* block

Hidden or "Back Door" Channels

An invisible "this" object is one example, but there are many cases where caller and callee procedures might communicate via some "hidden" channel — that is, parameters are passed in to functions without being explicitly notated in the list of arguments. This is one way to implement Design by Contract. Suppose a procedure takes two lists, which must be the same size. If there is indeed an explicit test like $\mathit{if}(x.\mathit{size}() == y.\mathit{size}())\{\ldots\}$ we can ensure that this condition is met within the "true" branch of the *if* (presuming, of course, the lists are not modified therein). So, the compiler could insert a "proof" object into the lexical scope of that branch, which may thereafter be silently passed to the DbC procedure. Conversely, if the programmer can guarantee that the same-size condition is met — e.g., if the second is built from the first preserving length — then they could explicitly notate such a proof in the current scope, thus enabling it to be similarly "back-doored".

Another example (a form of "Return Value Optimization") is passing pre-allocated (but not initialized) memory buffers for "placement new" when the caller cannot provide a default-initialized mutable reference and does not want a heap-allocated return value.

Interface Design Principles for the Research Object Context

What are some design patterns that should be encouraged?

- **Unify Pass-by-Reference Initialization and Pointer Return** Called procedures agnostic as to the source of initialized memory buffers/objects (whether pre-initialized reference arguments, delegating ownership of heap-allocated memory, or Return Value Optimization)
- **Programmer Control of Variable Lifetimes** Lifetimes can be associated with different lexical scopes or other variables'
- **"Overload by Contract"** Programmers supply different versions of a procedure depending on whether contract-tests are known at compile time to have been evaluated
- **Easy Foreign Function Interface** Calling native-compiled procedures should be possible with simple registration functions
- **Self-Documenting** Given $y = f(x)$, is y being initialized or re-initialized? Does f have side-effects? Was y default constructed?
- We can guarantee *const*. Can we *guarantee* to modify as with an lvalue?

Interface Design Principles for the Research Object Context

Assuming that a portion of Research Object code is a library developed in a scripting language specially engineered for data sets, we can then consider how type and procedural interfaces could be engineered for such a language. The design patterns need not simply reiterate "host" languages like C++. For example, a common question in C++ is whether to accept references or return pointers, for large/complex return values. A streamlined interface protocol could allow procedures to adopt both conventions — plus placement-new initialization mentioned above — depending on channel-package construction. Another interesting feature is "projecting" declared variables to take on the lifetime of either an inner, nested scope or outer, containing scope — which is particularly relevant when variables are declared in the course of aggregate expressions. Meanwhile, "Overload by Contract" is a way to implement DbC by requiring multiple versions of procedures, depending on the status of contract tests (pass, fail, or unknown). And intrinsic support for pattern-matching over graph-like or type-reflection structures is a technique for embedding query expressions directly into the language — including text-query for sentence objects.

Diamond Open Access

According to the Diamond (DOA) model, publications have neither Article Processing Charges (APCs) nor paywalls

- **Authors Retain Copyright** No fees for either authors or readers. Authors can share their work in any form (PDF, XML, HTML, etc.)
- **Unify Publications and Data Sets** Once manuscripts have the same licences as associated Research Objects, both PDF and machine-readable text representations of the written documents can be freely distributed within open-access data sets.
  - **This makes PDF customization and embedded full-text search possible**
- **Give Authors Greater Control** To compensate for missing APC revenue, publishers can reduce their workload per publication by delegating more responsibility to authors. This can be to authors' benefit and could streamline the document-preparation process.
- **Option to Retain Outside Help** If authors do want assistance with their manuscript, funds otherwise marked for APCs might instead be directed to specialists who could perform work needed to prepare a manuscript for publications. Rather than copy or digital editing alone, these specialists can take on the more complex tasks implied by Executable Research Objects (like writing dataset code, preparing raw data files, etc.).

Diamond Open Access

The "Diamond" Open Access model assumes that no fees are charged for either authors or readers. Often, authors retain full rights and control over their work. This model is frequently endorsed in terms of fairness and inclusion, but it has technological dimensions as well: programming data sets and Executable Research Objects is much easier when the textual and raw-data content shares the same licence and copyright. In particular, ideally Research Objects will include PDF versions of associated publications — and may have a custom PDF viewer with special features just for that paper — as well as text encoding of the same content in formats like JATS or serialized sentence objects. Assuming everything is above-board, this is untenable unless text documents and data sets share the same license permissions.

In lieu of Article Processing Charges, authors might take on more document-preparation responsibilities for themselves, or collaborate with others who are specialists in digital editing and related skills. This can be appropriate when preparatory work involves not just text manuscripts but also data sets and Research Object code.

Translational Science and Translational Medicine

Emphasizing the social benefits of scientific breakthroughs

A Rigorous Theory Although most science seeks positive impact, specialists in Translational Science/Medicine have developed detailed models of the stages research undertakes during the translational process

Research Object Foundations The code and protocols internal to research objects can lay the basis for on-site deployments and their digital/computational requirements

Emphasizing Community Outreach For community-oriented and/or public health initiatives, scientists may need to partner with social workers, language interpreters, experts in local culture, and potentially legal or economic advisers.

Scholarship "en situ" Observations from those directly working with community members and translational-science stakeholders may be just as valuable as less hands-on academic researchers. Publishers should encourage a pipeline for books and articles from the on-site, community/nonprofit context parallel to the ecosystem of institutional academic research.

Translational Science and Translational Medicine

The notion of "Translational Science" originated with "Translational Medicine", or "bench to bedside": how to transform research breakthroughs into clinical interventions that improve patient outcomes and experience. Beyond just medicine and biology, we now have "translational chemistry", "translational engineering", "translational sustainability", "translational humanities", and so forth. This is relevant for publishing because it affects the makeup of research teams: translational emphases imply diverse, interdisciplinary groups including some individuals whose specializations involve concrete implementation of research work, which can be affected by social, cultural, linguistic, economic, and logistical factors. Traslationality can imply people working "on site" for data acquisition, community outreach, language interpreters, nursing or social work, etc. Within such interdisciplinary teams, it is possible for one or more individuals to take particular responsibility for curating data sets and research publications — instead of deferring to publishers' APC-driven internal workflows or hiring external copy/digital editors. Publishers should also adjust to the social realities of a genre of scholarship reflecting practical, community-based experience and observations as well as academic research in labs and libraries.

Science/Humanities Integration

Software Language Engineering has a humanities dimension

- Source code does not "communicate with" computers, but there is still a linguistic dimension to how programmers' intentions are translated to code.
- The processing pipeline for compilers and Virtual Machines is arguably closer to linguistics than to mathematics

Executable Research Objects and the humanities

Translation to social benefit relies on cultural knowledge specific to humanities fields

- While raw data itself depends on narrow scientific disciplines, the organization and dissemination of data packages implicates humanities themes such as intertextuality and Philosophy of Science
- Humanities scholars may be especially sensitive to the nuances of written documents, rhetoric/argumentation, and sentence object modeling

Science/Humanities Integration

With "Translational Science" as a foundation, let's reiterate the principle that natural sciences and humanities are intrinsically linked in some places, and they reflect a continuum of overlapping disciplines more than a sharp divide. Many aspects of computer science, in particular, embody humanistic as well as scientific themes. Computer programming languages, for example, encapsulate some principles of natural language. The two are indeed very different — my point is not that linguists should help design programming languages because people somehow "talk to" computers — but still, the criteria for how computer languages are clear and expressive to human coders, and how they can be internally implemented, involve forms of structural reasoning characteristic of linguistics and other humanistic disciplines more than empirical sciences and mathematics. Humanities scholars should be encouraged to embrace digital/computational disciplines related to programming and data curation, and in this vein may become specialists in document preparation with rigorous computational skills as well as being adept at writing, editing, and other prerequisites for traditional publishing. Interdisciplinary collaborations and academic/community partnerships could provide joint efforts to buttress DOA models.

Thanks

Nathaniel Christen:

Thanks for Listening

This slide deck can be seen at https://SignScape.github.io/PNP/documents/Nathaniel_Christen_JATS-Con-2026-slides.pdf

The original paper is https://SignScape.github.io/PNP/documents/A-perspective-from-compiler-theory.pdf

The following page (github repository) has links to other formats for this manuscript and to several other, related documents: https://github.com/SignScape/PNP

Author's email: osrworkshops@gmail.com

Thanks

Thanks for listening!

If you would like to look at these slides again, I have a URL for them on the previous slide. Interleaved with that document is a parallel set of slides with versions of the comments I've made during the presentation (i.e., "textovers"). [Note: in the current document, the two groups of slides have been merged/interleaved together]

The prior slide also shows a link to my paper for this presentation. That document is available in several formats, and there are additional links on its first page.

If you are interested in looking at the demo code, that link is also available from the paper, as a github repository.